

noted - Tutorial #1 Report: Data Versioning and Experiment Tracking

Course: Engineering of Intelligent Models (EMI)

Group: EMI-3

Students: António Cruz (140129), Bruno Santos (140586), Pedro Miranda (129268), Ricardo Kayseller (95813)

Date: March 15th, 2026

1. Introduction

This report presents the first incremental delivery of noted, a collaborative, web-based MLOps workbench that we are developing as part of this project. noted aims to unify the full machine learning lifecycle - data versioning, experiment tracking, configuration management, pipeline orchestration, and model serving - into a single, integrated environment, eliminating the tool fragmentation that characterizes current MLOps workflows.

1.1 Problem Statement

Modern MLOps requires practitioners to operate across a fragmented landscape: data versioning happens in DVC, experiment tracking requires the MLflow UI, orchestration requires the Airflow UI, and object storage is managed through the MinIO console. Each tool has its own interface, authentication model, and mental model. Practitioners spend significant time context-switching between browser tabs, terminals, and dashboards - and configuration drift between experimentation and deployment is a persistent source of production failures.

1.2 Proposed Solution

We are building noted to address this fragmentation. The platform runs entirely within a Docker Compose stack and provides a browser-based interface where users can manage data, run experiments, configure pipelines, and deploy models without leaving the application. Rather than aggregating existing tool UIs into a dashboard, noted provides purpose-built views that communicate with backend services through their APIs. The underlying tools - DVC, MLflow, Hydra, Airflow, MinIO - remain the engines; noted is the cockpit.

1.3 Scope of This Delivery

For the use case that accompanies the platform development, we apply noted to a weather forecasting problem using the Jena Climate dataset, demonstrating how the platform supports the end-to-end MLOps workflow. This Tutorial #1 delivery focuses on the foundational layers that we have implemented so far: data versioning (DVC + MinIO) and experiment tracking (MLflow). Subsequent deliveries will progressively add configuration management (Hydra), pipeline orchestration (Airflow), model registry, model serving (FastAPI/LitServe), and a standalone prediction frontend.

2. Platform Design

2.1 Architecture

We designed noted as a multi-service Docker Compose stack comprising 13 containers that work together as a unified platform. The main `noted` container hosts a FastAPI backend and serves a vanilla ES6 frontend (no build step, no framework). It is supported by dedicated containers for MLflow (`noted-mlflow`), MinIO (`noted-minio`), PostgreSQL (`noted-postgres`), Redis (`noted-redis`), and a full Apache Airflow cluster (API server, scheduler, DAG processor, worker, triggerer). An nginx reverse proxy sits in front, routing all services under a single origin - MLflow at `/mlflow` , Airflow at `/airflow` , MinIO at `/minio` - so the user accesses everything through one URL and secrets never reach the browser.

The UI follows a VS Code-like 4-column layout that we implemented to support the density of features planned across all project phases:

- Icon Bar - vertical navigation strip for switching between workspace views
- Sidebar (Workspace Explorer) - collapsible panel hosting the project tree, experiments browser, storage browser, source control, and virtual environment manager
- Center Pane - tabbed content area supporting notebooks, Python file editors, service UIs (MLflow, Airflow, MinIO as embedded iframes), and detail views
- Right Panel - AI assistant chat (planned for future phases)

2.2 Design Principles

We established a set of architecture design principles that guide all implementation decisions:

- Zero Vendor Lock-In - all artifacts produced within noted are standard and portable. Notebooks are standard `.ipynb` files. MLflow runs use the standard tracking API. DVC pointer files are standard `.dvc` YAML. Airflow DAGs use standard operators only. A user

can eject from noted at any time and continue working with the same tools from the command line.

- Backend Services Stay Canonical - MLflow remains the source of truth for experiments. DVC for data versions. Airflow for pipeline execution. noted reads from and writes to these services but never duplicates their state.
- Integration Over Aggregation - noted is not an iframe wrapper. We build purpose-built views - the experiment browser, the storage panel, the source control view - that surface the information practitioners need without requiring them to navigate to each service's native UI.
- Explicit Over Magical - when noted performs an action on the user's behalf (injecting MLflow tracking context, committing a DVC pointer file), the action is visible and tagged. Automatic instrumentation is opt-in and transparent.
- Progressive Complexity - a new user can open noted, create a notebook, and execute Python cells - exactly like Google Colab. As their needs grow, they discover data versioning, experiment tracking, and orchestration features organically.

2.3 Infrastructure Stack

Service	Container	Role
noted	noted	Application server (FastAPI + frontend)
MLflow	noted-mlflow	Experiment tracking, model registry
Airflow	noted-airflow-apiserver	Pipeline orchestration (REST API)
MinIO	noted-minio	S3-compatible object storage (DVC remote + MLflow artifacts)
PostgreSQL	noted-postgres	Metadata store (MLflow, Airflow, noted)
Redis	noted-redis	Celery broker for Airflow workers
nginx	noted-nginx	Reverse proxy, unified routing

3. Dataset Description

The chosen dataset is the Jena Climate dataset, a multivariate meteorological time series recorded by the Max Planck Institute for Biogeochemistry in Jena, Germany. It contains atmospheric measurements collected every 10 minutes over the period from January 2009 to December 2016, totaling approximately 420,000 observations across 15 columns (14 meteorological variables plus a timestamp).

#	Feature	Example	Description
1	Date Time	01.01.2009 00:10:00	Date-time reference
2	p (mbar)	996.52	Atmospheric pressure
3	T (degC)	-8.02	Air temperature
4	Tpot (K)	265.4	Potential temperature
5	Tdew (degC)	-8.9	Dew point temperature
6	rh (%)	93.3	Relative humidity
7	VPmax (mbar)	3.33	Saturation vapor pressure
8	VPact (mbar)	3.11	Actual vapor pressure
9	VPdef (mbar)	0.22	Vapor pressure deficit
10	sh (g/kg)	1.94	Specific humidity
11	H2OC (mmol/mol)	3.12	Water vapor concentration
12	rho (g/m**3)	1307.75	Air density
13	wv (m/s)	1.03	Wind speed
14	max. wv (m/s)	1.75	Maximum wind speed
15	wd (deg)	152.3	Wind direction

Following the project guidelines, which recommend avoiding near-deterministic relationships with the target variable, a subset of 6 input features was selected for modelling: T (degC), p (mbar), rh (%), wv (m/s), max. wv (m/s), and wd (deg).

Within noted, the raw dataset is managed via DVC with a MinIO (S3-compatible) remote storage backend. DVC assigns a cryptographic MD5 hash to each tracked file, enabling exact reproduction of any prior data state. This hash is logged as an MLflow parameter in every experiment run, establishing strict data lineage - a core traceability requirement that links every model evaluation to the precise dataset version used during training.

Preprocessing includes resampling from 10-minute to hourly frequency (mean aggregation), replacement of erroneous wind speed values (-9999.0), removal of duplicate rows (~327 affected), and cyclical encoding of wind direction into sine/cosine components. The resulting dataset is split temporally into training, validation, and test sets to prevent information leakage.

4. Machine Learning Problem

The forecasting task is formulated as a multivariate-input, univariate-output, multi-step regression problem: given a sliding window of $L = 120$ hours (5 days) of past meteorological observations across the selected input features, the model predicts the air temperature (T) for the next $H = 24$ hours in a single forward pass (direct multi-step strategy).

Aspect	Detail
Target variable	Air temperature - T (degC)
Input window	$L = 120$ hours (5 days) of multivariate observations
Forecast horizon	$H = 24$ hours ahead
Strategy	Direct multi-step (single forward pass produces full horizon)
Evaluation metrics	MAE and RMSE, computed globally and per-horizon (1h to 24h)
Baseline	Persistence model (future temperature = last observed value)

Two model architectures are compared:

Model	Type	Description
GRU	Recurrent	Gated Recurrent Unit network. Processes the full input sequence recurrently to capture temporal dependencies. Optimized via Optuna hyperparameter search over hidden dimensions, number of layers, learning rate, dropout, and optimizer.
PatchTST	Transformer	Patch Time Series Transformer. Segments the input window into fixed-length patches, embeds them as tokens, and applies stacked Transformer encoder blocks with multi-head self-attention to model both short- and long-range temporal interactions. Optimized via Optuna over patch size, embedding dimension, number of heads, encoder depth, and feed-forward dimension.

Both architectures are compared against the persistence baseline across all 24 forecast horizons. Results show MAE starting at approximately 0.6°C for 1-hour-ahead predictions, gradually increasing to around $2.1\text{--}2.2^{\circ}\text{C}$ at the 24-hour horizon - significantly outperforming the naive baseline, which deteriorates rapidly beyond short-term horizons.

All experiments - parameters, metrics, model artifacts, and data version hashes - are tracked in noted's integrated MLflow server, enabling systematic comparison across architectures and

configurations directly from the platform's experiment browser.

5. Implemented Capabilities (Tutorial #1)

This delivery demonstrates the features we have built so far, corresponding to noted's Phase 1A (completed) and Phase 1B (in progress).

5.1 Notebook Environment

We built a notebook editor that supports the full interactive workflow required for data exploration and model training:

- CodeMirror 6 editor with Python syntax highlighting and dark theme
- Cell execution via Jupyter kernels (Shift+Enter, Ctrl+Enter, Run All)
- Streamed output rendering (stdout, stderr, images, error tracebacks)
- Markdown cells for documentation (edit/preview toggle)
- Drag-and-drop cell reordering
- Multiple Python runtime support (3.10–3.14) with per-project virtual environments
- GPU acceleration support (CUDA runtime passthrough)

5.2 Data Versioning (DVC + MinIO)

We integrated DVC data versioning directly into the workspace:

- Source Control panel with Git and DVC sections side by side. DVC-tracked files show push/pull status with dedicated buttons for syncing with the MinIO remote.
- VS Code-style tree decorations on workspace nodes - colored dots (teal for DVC-tracked), status letter badges (T for tracked, M for changed, N for new), and "DVC" source badges distinguishing DVC-managed files from Git-managed files.
- Context menu on data files: right-click → "Track with DVC" runs `dvc add` in the background, creates the `.dvc` pointer file, and updates the tree decorations immediately.
- Version history in the detail pane showing the complete history of a `.dvc` file (hash, commit date, file size) by walking `git log` on the pointer file.

5.3 Experiment Tracking (MLflow)

We implemented seamless MLflow integration requiring no configuration from the user:

- `MLFLOW_TRACKING_URI` and `MLFLOW_EXPERIMENT_NAME` are automatically injected into every kernel session at startup. Users can `import mlflow` and start logging immediately.
- Experiments section in the workspace tree provides a browsable experiment/run hierarchy with lazy loading. Run status icons (green check for completed, spinner for running, red X

for failed) give immediate visual feedback. Clicking a run opens a detail panel showing metrics, parameters, tags, and artifacts.

- Run Manager is a visual tool for defining named cell groups as MLflow run templates. Users assign cells to a run by clicking, and colored cell badges indicate run membership. "Execute Run" runs all assigned cells sequentially wrapped in a single MLflow run with automatic `mlflow.start_run()` / `mlflow.end_run()` injection, providing a code-free way to structure experiments.
- Auto-instrumentation engine provides optional silent pre/post code injection that starts an MLflow run before cell execution and activates framework-specific autolog (sklearn, PyTorch, TensorFlow, XGBoost, LightGBM) after execution. Back-off logic skips injection if the user writes their own `mlflow.start_run()`.
- Storage browser presents the MinIO bucket/object tree in the workspace sidebar, allowing direct inspection of MLflow artifacts and DVC-pushed data files.

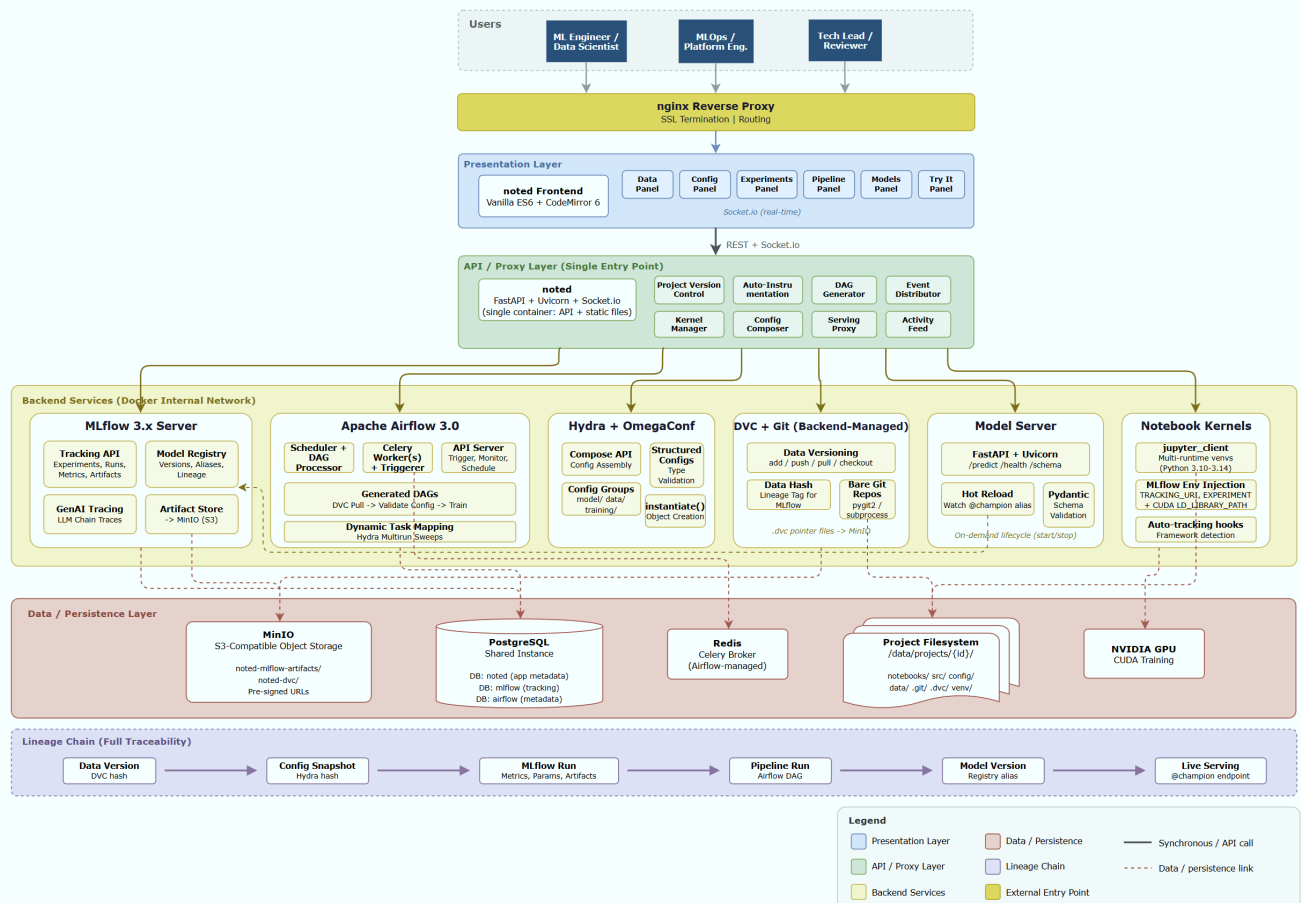
5.4 Source Control (Git)

We built full Git integration via the Source Control sidebar:

- Branch management: create, switch, list branches
- Tag management: create, list, delete tags
- Commit workflow: staged/unstaged file list with status badges, commit message, push/pull with ahead/behind indicators
- Diff viewer: expandable commit items showing per-file diffs
- GitHub integration via PAT-based remote authentication

6. Architecture Diagram

The following diagram illustrates noted's high-level technical architecture, including all platform layers and backend service integrations:



7. Next Steps

Building on the data versioning and experiment tracking foundation delivered here, we plan to extend noted with the following capabilities in upcoming deliveries:

- **Configuration management** - a Hydra integration providing form-based config group selection (e.g., switching between GRU and Transformer architectures), CLI override builder, and resolved config preview, eliminating the need to manually edit YAML files or construct command-line overrides.
- **Pipeline orchestration** - an Airflow integration enabling users to define, trigger, and monitor multi-step DAGs (ingestion → preprocessing → training → evaluation) directly from noted's workspace, with parameterized trigger forms generated from DAG parameter schemas.
- **Model governance and serving** - MLflow Model Registry integration for version management and promotion workflows (staging → production), FastAPI/LitServe model serving for real-time predictions, and a standalone prediction frontend for end-user interaction with deployed models.